Phyelds: A Pythonic Framework for Aggregate Computing

Gianluca Aguzzi[0000−0002−1553−4561], Davide Domini[0009−0006−8337−8990], Nicolas Farabegoli[0000−0002−7321−358X], and Mirko Viroli[0000−0003−2702−5702]

University of Bologna, Bologna, Italy
{gianluca.aguzzi, davide.domini, nicolas.farabegoli, mirko.viroli}@unibo.it

Abstract. Aggregate programming is a field-based coordination paradigm with over a decade of exploration and successful applications across domains including sensor networks, robotics, and IoT, with implementations in various programming languages, such as Protelis, ScaFi (Scala), and FCPP (C++). A recent research direction integrates machine learning with aggregate computing, aiming to support large-scale distributed learning and provide new abstractions for implementing learning algorithms. However, existing implementations do not target data science practitioners, who predominantly work in Python—the de facto language for data science and machine learning, with a rich and mature ecosystem. Python also offers advantages for other use cases, such as education and robotics (e.g., via ROS). To address this gap, we present Phyelds, a Python library for aggregate programming. Phyelds offers a fully featured yet lightweight implementation of the field calculus model of computation, featuring a Pythonic API and an architecture designed for seamless integration with Python's machine learning ecosystem. We describe the design and implementation of Phyelds and illustrate its versatility across domains, from well-known aggregate computing patterns to federated learning coordination and integration with a widely used multi-agent reinforcement learning simulator.

Keywords: Aggregate programming · Distributed machine learning · Field-based coordination

1 Introduction

The proliferation of distributed computing systems—from sensor networks and IoT infrastructures to robotic swarms and edge-cloud continua—has created an urgent need for programming abstractions that can effectively manage collective behavior at scale [12]. Traditional approaches to distributed system development focus on individual devices, requiring programmers to manually orchestrate communication, synchronization, and coordination among potentially thousands of heterogeneous nodes. This device-centric perspective becomes increasingly untenable as systems grow in scale and complexity, motivating the emergence of

macroprogramming paradigms [13] that shift the focus from individual devices to the collective behavior of the system as a whole.

Among macroprogramming approaches, aggregate programming [10] has gained significant attention due to its principled foundation in the *field calculus* [8]—a minimal functional language that models computation as the manipulation of *computational fields*, where values are distributed across space and evolve over time. Application domains span sensor networks, swarm robotics, smart cities, and IoT systems, where the ability to reason about global behavior while ensuring local execution is paramount.

A promising research direction integrates aggregate programming with machine learning [4]. This synergy is bidirectional: aggregate programming facilitates scalable distributed learning—e.g., in federated learning and multi-agent reinforcement learning [21,5,19]—while machine learning enables data-driven adaptation within aggregate programs [3]. However, realizing this potential faces a significant practical barrier: existing implementations of aggregate programming—such as Protelis [33], ScaFi [15], and FCPP [7]—target languages (Java, Scala, and C++, respectively) that are not commonly used by machine learning practitioners. Python has become the *de facto* standard for data science and machine learning, boasting a rich ecosystem of libraries including TensorFlow [1], PyTorch [30], and scikit-learn [31]. The absence of a Python-based aggregate programming framework thus hinders the adoption of this paradigm in machine learning contexts and limits the potential for cross-pollination between these research communities.

To bridge this gap, we present Phyelds[1], a lightweight and Pythonic library for aggregate programming. Phyelds delivers a compact but comprehensive implementation of the field calculus, offering an API that embraces Python's idioms and conventions. Unlike its functional predecessors, Phyelds adopts an imperative and object-oriented programming style that is more familiar to the majority of Python developers, lowering the barrier to entry for newcomers to aggregate programming. The framework is designed with seamless integration in mind, providing bindings for popular machine learning libraries such as TensorFlow and PyTorch, as well as multi-agent reinforcement learning environments like VMAS [11]. We demonstrate its applicability through well-known aggregate computing patterns, self-organizing federated learning [21,17,20], and multi-agent reinforcement learning via integration with the VMAS simulator [11], where aggregate programming abstractions are leveraged to coordinate collective agent behavior.

Beyond machine learning, bringing aggregate programming into the Python ecosystem opens opportunities in other Python-dominated domains. In **education**, Python's gentle learning curve makes it an ideal vehicle for introducing students to collective programming concepts. In **robotics**, integration with ROS (Robot Operating System) [24] enables the deployment of aggregate programs on physical robot swarms.

[1] `https://github.com/phyelds/phyelds`

The remainder of this paper is structured as follows. Section 2 provides a brief overview of aggregate programming, covering the system model (Section 2.1), the field calculus operators (Section 2.2), and the alignment mechanism (Section 2.3). Section 3 presents the design and implementation of PHYELDS. Section 4 illustrates the library through concrete examples, including common aggregate computing patterns, self-organizing federated learning, and integration with the VMAS multi-agent reinforcement learning simulator. Finally, Section 5 concludes with directions for future work.

2 Aggregate Programming - A Brief Overview

Aggregate computing (AC) [10] is a macroprogramming paradigm for the development of distributed systems, where the focus is on the collective behavior of a system of devices, rather than on the individual behavior of each device. Starting from natural inspiration, the computation is expressed as a manipulation over *computational fields*, where the value of a field at a given point in space and time represents the state of the device located there. The field calculus [10] is a minimal functional language that captures the essence of aggregate programming, providing a formal model for reasoning about field computations.

2.1 System Model

Typically, an aggregate computing system consists of a collection of devices, each equipped with local computation capabilities, sensors for perceiving the environment, and communication interfaces for exchanging information with neighboring devices. Each device is connected to a subset of other devices, forming a dynamic and potentially unreliable network topology. The communication model is typically based on message passing, where devices can send and receive messages asynchronously to and from their neighbors.

Each device executes the same aggregate program, which can be conceptually viewed as a function that takes the device's local state and the values received from neighbors as input, and produces an output value that represents the device's contribution to the global computation. Specifically, each device organizes its execution into discrete rounds, where in each round it performs the following steps (Figure 1):

1. **Sense**: the device collects the neighbors' messages, reads its sensors, and gathers the previous local state;
2. **Compute**: the device executes the aggregate program using the collected information, producing a new local state and an output value;
3. **Interact**: the device sends the output value to its neighbors, and stores the new local state for the next round.

The rounds are not globally synchronized, and they typically proceed asynchronously across devices. This execution model is resilient to topology changes, communication and device failures, relying on an eventual behavior, which is the foundational property of self-stabilization.

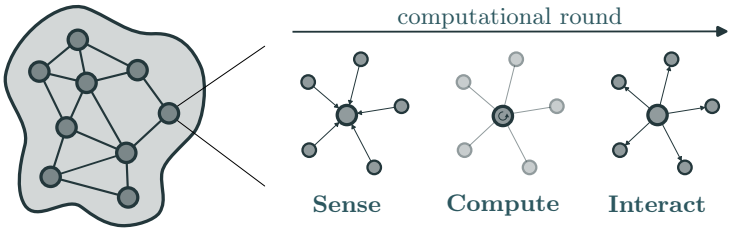


Fig. 1: Graphical representation of the aggregate computing system model.

2.2 Field Calculus Operators

The field calculus provides a small set of core operators that enable the expression of complex distributed computations in a concise and composable manner. In what follows, we briefly describe the main operators of the field calculus.

rep defines stateful computation among device rounds, allowing each device to maintain and evolve a local state. Evaluates an initial value during the first round and then updates it in subsequent rounds based on the previous state and an evolving function.

nbr enables the exchange of information between neighboring devices, building a *neighborhood field* that captures the values of neighboring devices aligned with that expression.

if provides conditional branching, allowing devices to diverge in their execution paths based on local conditions. Only the subset of neighboring devices aligned with the same branch condition are visible inside the branch.

foldhood aggregates information from neighboring devices using a specified binary operator, enabling the computation of collective properties such as sums, counts, or maxima across the neighborhood.

On top of these core operators, a rich library of higher-level constructs can be built, including gradient computation, information spreading and collection, gossip protocols, and leader election algorithms.

2.3 Alignment Mechanisms

The alignment is a fundamental aspect of aggregate programming, ensuring that devices evaluating the same program are considered (aligned) and can correctly exchange information. We refer to *aligned devices* as those that satisfy the following two conditions: (i) the devices are executing the exact same program; (ii) the produced output (i.e., the value tree) resulting from the execution, reflects the same sequence of function calls and operator applications. Any violation of these conditions results in *misalignment*, where the devices are not considered part of the same computation.

When an aggregate program is executed, two devices may be aligned for some parts of the program and misaligned for others. Consider a program where an `if`

operation is used. Based on the boolean condition evaluated at runtime, some devices may execute the `then` branch, while others execute the `else` branch. This divergence at runtime leads to *intended* misalignment, where on the two branches, only a subset of devices is aligned with each other. Once the execution of the branches is completed, all the devices will realign as they will converge to the same program point again. The reconciliation of the devices is because of the assumption that all the devices execute the same program, and thus they will eventually converge to the same program point again, despite the divergence at runtime.

In practice, the alignment mechanism is implemented by associating a unique identifier to each aggregate operator which requires either a stateful operation (e.g., `rep`), a neighbor interaction (e.g., `nbr`) or a network partitioning (e.g., `if`).

3 The Phyelds Library

PHYELDS was conceived with three key design goals. First, we provide an **API** that is idiomatic to Python, embracing its object-oriented and imperative programming style to lower the barrier to entry for developers. First, existing frameworks are predominantly based on functional languages and adopting an imperative style is crucial to make aggregate programming accessible to the majority of Python users. Second, the library relies on a **lightweight and modular architecture** designed for applicability across diverse contexts, ranging from small-scale simulations and machine learning applications to deployment on physical devices. To support this versatility, users can selectively incorporate only the components relevant to their specific use case. Third, we ensure **integration with the Python ecosystem** by providing a simple execution framework for verifying the behavior of implemented blocks. Additionally, we provide bindings to popular machine learning libraries, multi-agent reinforcement learning environments, and robotics frameworks.

With these goals in mind, the architecture of PHYELDS is structured into distinct modules, each responsible for a specific aspect of the aggregate programming model. Namely, it comprises five principal components (Figure 2): the `VM`, the `Data`, the `Calculus`, the `Library`, and the `Simulator` modules.

VM Module. The VM module governs the execution of aggregate programs. It provides the low-level APIs required to track device states, manage function calls for the alignment mechanism, and maintain execution contexts. Specifically, this module handles gathering messages from neighbors, persisting device state across rounds, and orchestrating the execution of field calculus operators. The principal classes in this module are `Engine`, `NodeContext`, and `EngineState`. The `Engine` class embodies the core logic of the virtual machine and serves as the interface between the VM and the rest of the library. To use the engine, it must be initialized with messages received from neighbors, the previous state, the device identifier, and the device context (encapsulating time, sensor readings, and other contextual data). Subsequently, the aggregate program is executed,

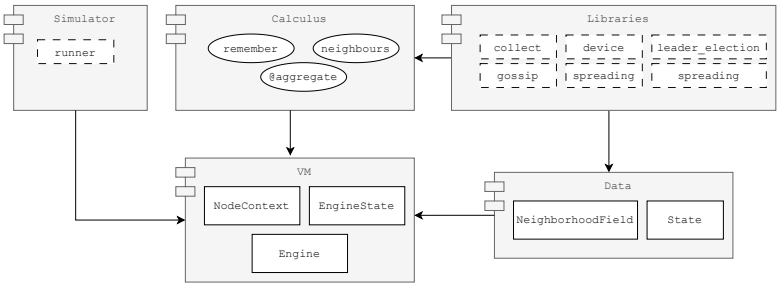


Fig. 2: Architecture of Phyelds. Modules (components) contain classes (rectangles), submodules (dashed borders), and functions (ovals). Arrows indicate dependencies.

leveraging this information to perform computations and generate a new state alongside messages for neighboring devices. Finally, the engine is reset to prepare for the next round. The typical usage pattern is illustrated in Listing 1.1:

```
from phyelds import engine
from phyelds.vm import Engine
# 1. Setup: initialize the engine with the current context
engine.get().setup(
    node_context=node_ctx,
    messages=received_messages,
    state=previous_state
)
# 2. Execute the aggregate program
result = my_aggregate_program()
# 3. Cooldown: reset engine and retrieve messages to send
new_state, messages_to_send = engine.get().cooldown()
```

Listing 1.1: Engine usage pattern.

Data Module. The Data module contains the fundamental data structures representing computational fields. The two principal components are the `State` and `NeighborhoodField` classes. The `State` class maintains a device's state across rounds, corresponding to the `rep` construct in traditional field calculus. Implemented as a proxy object, it allows manipulation as a standard Python object. Additionally, the `State` class stores the usage path to ensure the alignment mechanism functions correctly. The `NeighborhoodField` class represents values received from neighboring devices, corresponding to the `nbr` construct. Although internally implemented as a dictionary mapping device identifiers to values, this detail is abstracted from the user, who interacts with it as a standard Python object. The class supports operations such as element addition, arithmetic operations between neighboring fields, and field merging. Instead of exposing an ad-hoc `foldhood` operator, aggregation over neighbors is achieved through standard Python constructs such as list comprehensions and built-in functions like `sum` or `max`.

Calculus Module. The Calculus module provides the primary API for writing aggregate programs, encompassing the core operators and constructs of the field calculus. The central element is the `@aggregate` decorator, which transforms a standard Python function into one executable within an aggregate context. Annotated functions undergo a transformation as illustrated in Listing 1.2:

```
def aggregate(func):
    def wrapper(*args, **kwargs):
        # Enter the execution context
        engine.get().enter(func.__name__)
        # Apply code transformations if necessary
        result = transform_code(func)(*args, **kwargs)
        # Exit the execution context
        engine.get().exit()
        return result
    return wrapper
# User-defined aggregate program
@aggregate
def my_program(sensors):
    return sensors["temperature"] + 1
```

Listing 1.2: Transformation of an `@aggregate` function.

The `transform_code` function is responsible for managing alignment. Under the hood, it parses the function's source code into an Abstract Syntax Tree (AST) and applies a node transformer to intercept `if` statements. To ensure the engine can track and match execution paths across devices, the transformer rewrites the code by wrapping the `if` and `else` bodies within explicit `align_left()` and `align_right()` context managers. Listing 1.3 shows an example of how this transformation is applied.

```
# Original user code
if condition:
    do_something()
else:
    do_other()
# Transformed code executed by the engine
if condition:
    with align_left():
        do_something()
else:
    with align_right():
        do_other()
```

Listing 1.3: Example of code transformation for conditionals.

The `engine` object is a globally accessible instance containing the current VM. It is initialized prior to program execution and reset after each round. To support parallel execution (e.g., in simulation contexts), this object is defined as a context variable (Listing 1.4):

```
from contextvars import ContextVar
from phyelds.vm import Engine
engine: ContextVar[Engine] = ContextVar("engine")
```

Listing 1.4: Engine as a context variable.

The other two fundamental operators are `remember` and `neighbors`, which implement the `rep` and `nbr` constructs of the field calculus. The `remember` operator

maintains state across rounds. In standard field calculus, `rep` is a functional operator that takes an initial value and an update function:

```
rep(0) { (value) => value + 1 }
```

While this style is native to functional languages, it is often less familiar to Python developers. Therefore, inspired by React's state management, calling `remember` returns a tuple containing a setter function and the current value, clearly separating state updates from value access:

```
set_value, value = remember(0)
set_value(value + 1)  # Imperative state update
```

Listing 1.5: Using the `remember` construct.

The `neighbors` construct facilitates interaction with neighboring devices. It takes a value, transmits it to neighbors, and returns the values received from neighbors as a `NeighborhoodField`:

```
nbr_values = neighbors(my_value)  # Returns NeighborhoodField
```

Listing 1.6: Using the `neighbors` construct.

When the argument of the `neighbors` function is a `State` object, transmission occurs lazily: the `State` value is only sent if it has changed since the previous round, enabling efficient tracking of state updates.

Library Module. The Library module builds upon the Calculus module, providing a collection of higher-level constructs and building blocks for common aggregate programming patterns. These blocks implement well-established algorithms from aggregate programming literature, enabling developers to compose complex distributed behaviors from reusable components.

The module is organized into several sub-modules, each addressing a specific aspect of aggregate computation. The `device` sub-module accesses device-specific information such as the local identifier (`local_id`), spatial coordinates (`local_position`), and sensor readings (`sense`). The `time` sub-module offers temporal primitives including current simulation time, a round counter, and decay functions for time-dependent behaviors. The `distances` sub-module computes inter-device distances using either Euclidean metrics based on positions or hop-count metrics for topology-based reasoning. The `spreading` sub-module implements information dissemination patterns. Its core function, `distance_to`, computes a gradient (potential field) emanating from designated source nodes. This serves as the foundation for higher-level operations such as `broadcast`, which propagates data outward from sources, and `cast_from`, which applies accumulation functions along the propagation path. Complementing spreading, the `collect` sub-module implements convergecast patterns that aggregate information toward source nodes along gradient paths. The `find_parent` function establishes parent-child relationships based on potential values, while `collect_with` enables flexible aggregation using user-defined accumulation functions. Specialized variants include `count_nodes` for counting

devices in sub-regions and `sum_values` for numeric aggregation. The `gossip` sub-module provides epidemic-style communication protocols for network-wide information sharing. The generic `gossip` function spreads values across the network using a user-specified combination operator, with convenience functions `gossip_max` and `gossip_min` for computing global extrema. A self-stabilizing variant, `stabilizing_gossip`, bounds propagation by the network diameter to ensure convergence. Finally, the `leader_election` module implements a well-known self-stabilizing algorithm [29] via the `elect_leaders` function, using random identifiers to break symmetry. It ensures that every device is within a specified distance d from exactly one leader, creating a stable Voronoi partition that converges regardless of the initial state. More details on the algorithm can be found in the original paper [29] and in further extensions [28,27].

These building blocks can be composed to implement complex distributed algorithms. For instance, combining leader election with gradient computation and collection enables hierarchical data aggregation systems where elected leaders collect and summarize sensor readings from their respective regions. Detailed usage examples are provided in Section 4.

Simulator Module. The Simulator module provides a framework for testing and debugging aggregate programs in a controlled environment. It enables users to create virtual networks of devices, define their communication, and observe program execution over multiple rounds.

The module is built around three core abstractions. The `Node` class represents an individual device characterized by a unique identifier, spatial position, and arbitrary sensor data. The `Environment` class manages the collection of nodes and, crucially, determines the neighborhood topology via a configurable neighborhood function. The `Simulator` class orchestrates execution using a discrete-event model, maintaining an event queue that schedules program execution across nodes. Neighborhood definition is a key aspect of the simulator design. Rather than hard-coding a specific topology, the module provides pluggable neighborhood functions to determine which nodes can communicate. Built-in options include `radius_neighborhood` (nodes within a specified Euclidean distance), `k_nearest_neighbors` (each node connects to its k closest peers), and `full_neighborhood` (fully connected network). Users can also define custom neighborhood functions to model arbitrary communication patterns or dynamic topologies. The module also provides deployment utilities for generating common spatial distributions, including regular grid layouts (`grid_generation`), perturbed lattices (`deformed_lattice`) for more realistic deployments, and random distributions within circular regions (`random_in_circle`). For observability, the simulator supports `Monitor` objects attached to the simulation. Monitors receive callbacks at the start, after each event, and at completion, enabling logging, visualization, or metric collection.

Additionally, the module integrates with the VMAS [11] framework through the `VmasEnvironment` class, enabling aggregate programs to control agents in physics-based multi-agent reinforcement learning scenarios. This integration al-

lows researchers to combine aggregate programming abstractions with learned policies for coordinated robot control.

4 Example of Use

In this section, we provide several examples of use of the PHYELDS library[2], demonstrating how to implement common aggregate programming patterns and how to integrate with machine learning frameworks for coordination-relevant learning scenarios. PHYELDS can be used in a normal Python environment, such as a Jupyter notebook[3], and it also provides a native simulator for testing and debugging aggregate programs.

Common Aggregate Computing Patterns. A common and well-established pattern in AC is the construction of a *channel* between a source device and a target device. This pattern is frequently employed to model information flows, logical routing structures, or spatial constraints within a distributed system.

A channel (Figure 3a) is defined as the set of devices that lie within a given distance from a minimum path connecting the source and the target. This distance represents the *width* of the channel, allowing the model to range from a single-device path to a wider region surrounding the optimal route. The final desired outcome is a boolean computational field, where devices belonging to the channel evaluate to `True`, while all others evaluate to `False`.

Listing 1.7 shows the PHYELDS code implementing this behavior. The first step consists of computing the computational field containing distances to neighboring devices, which is obtained through the library function `neighbors_distances` (line 3). Based on this field, the distance from the target device is then computed using the `distance_to` operator (line 4), with the source argument given by the `''target''` sensor, which evaluates to `True` exclusively on the target device. To construct the channel, all devices lying on the minimum path between the source and the target are collected (line 5). This is achieved by propagating information from the target distance field while constraining it with the `''source''` sensor, effectively identifying the devices belonging to the shortest path (line 6). Finally, to assign a channel width different from one (i.e., a single-device path), each device computes its distance from the minimal channel and is included in the final boolean field if this distance is smaller than the specified channel width (line 7).

```
1 @aggregate
2 def main():
3     distances = neighbors_distances()
4     target_distance = distance_to(sense("target"), distances)
5     nodes_in_path = collect_or(target_distance, sense("source"))
6     distance_from_path = distance_to(nodes_in_path, distances)
```

[2] https://github.com/domm99/artifact-coordination-2026-phyelds

[3] Please, take a look at the online binder repository at https://mybinder.org/v2/gh/phyelds/phyelds-examples/HEAD?urlpath=%2Fdoc%2Ftree%2F%2Fbinder%2Fphyelds-example.ipynb

```
    channel = 1.0 if distance_from_path < width else 0.0
    return channel
```

Listing 1.7: Construction of a channel from a source device to a target device.

Listing 1.8 shows how to set up and run the simulation for the channel example (also available in the online Binder notebook). A `Simulator` is first instantiated and configured with a radius-based neighborhood function (lines 1–2). A 20×20 deformed lattice topology is then created with positional noise to model a realistic device deployment (line 3). Each node is initialized with `source` and `target` sensor values set to `False`; the first node in the list is designated as the source and the last as the target (lines 5–10). The aggregate program (i.e., the `main` function) is scheduled on every node with a fixed time step of 0.1 seconds (lines 12–13). Finally, a `RenderMonitor` is attached to visualize the evolving field using colored nodes and drawn edges, saving the result to a video file, and the simulation is run for 10 seconds (lines 15–24).

```
1  simulator = Simulator()
2  simulator.environment.set_neighborhood_function(radius_neighborhood(0.12))
3  deformed_lattice(simulator, 20, 20, 0.1, 0.01)
4  # initialise sensor data on each node
5  for node in simulator.environment.nodes.values():
6      node.data = {"source": False, "target": False}
7  # designate source and target nodes
8  simulator.environment.node_list()[0].data["source"] = True
9  target = simulator.environment.node_list()[-1]
10 target.data["target"] = True
11 # schedule the aggregate program on every node
12 for node in simulator.environment.nodes.values():
13     simulator.schedule_event(0.0, aggregate_program_runner, simulator, 0.1, node, main)
14 # configure rendering and run
15 RenderMonitor(
16     simulator,
17     RenderConfig(
18         effects=[DrawEdges(), DrawNodes(color_from="result")],
19         mode=RenderMode.SHOW,
20         save_as="channel.mp4",
21         dt=0.1
22     )
23 )
24 simulator.run(10)
```

Listing 1.8: Simulation setup and execution for the channel example.

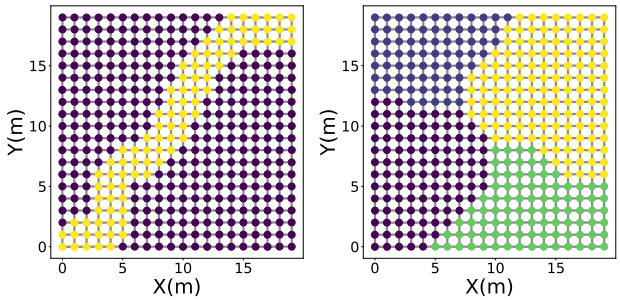
A second widely used pattern in aggregate computing is known as *Self-Organizing Coordination Regions* (SCR) [14]. This pattern is inspired by classical results in distributed systems [22,16,32], where large-scale systems are often partitioned into smaller subcomponents, each governed by a special node—referred to as a *leader*—responsible for coordination tasks. SCR enables the spatial partitioning of devices into disjoint subregions, effectively forming a Voronoi tessellation [9] of the space, and the election of a leader within each region. Once leaders have been established, each of them acts as the coordinator for its own region. In particular, non-leader devices funnel their local information toward the leader through a convergecast process, the leader computes a new result based on the collected data, and subsequently disseminates the updated information back to all devices in the region via a broadcast operation.

Listing 1.9 illustrates an instance of this pattern, where SCR is used to count the number of devices belonging to each region and to propagate the resulting count to all devices within the same region. This is achieved by first electing leaders (line 4), then assigning devices to regions according to minimum-distance potentials(line 5), and finally combining convergecast (line 6) and broadcast operations to aggregate (line 7) and disseminate information.

Although this example focuses on a simple aggregation task, the SCR pattern is general and can be applied in significantly more complex scenarios. For instance, it can be used to coordinate the training of a machine learning model independently within each region, as discussed in the following subsection.

```
1 @aggregate
2 def main():
3     distances = neighbors_distances()
4     leader = elect_leaders(4, distances)
5     potential = distance_to(leader, distances)
6     nodes = count_nodes(potential)
7     area_value = broadcast(leader, nodes, distances)
8     return area_value
```

Listing 1.9: Self-organizing Coordination Regions pattern.



(a) Channel example (b) SCR example

Fig. 3: Examples of common aggregate programming patterns implemented in Phyelds. On the left, a channel is constructed between a source device (bottom left) and a target device (top right). The channel color is yellow. On the right, devices are partitioned into coordination regions (different colors).

Self-Organizing Federated Learning. Federated Learning (FL) [25,26] is a distributed learning paradigm in which multiple devices collaboratively train machine learning models while keeping training data local and private. Self-Organizing Federated Learning (SOFL) [17,21] extends this paradigm by removing centralized coordination and enabling devices to autonomously form multiple federations, each learning a specialized model that better reflects local data characteristics. This is achieved by combining clustered federated learning with

self-organizing coordination mechanisms, allowing the system to adapt dynamically to spatial deployment and non-Identicaly and independently distributed (non-IID) data, specially when heterogeneity comes from spatial distribution of partecipating devices [18].

Listing 1.10 shows an implementation of a SOFL client using PHYELDS. Each device interleaves local training with self-organizing coordination. The computation starts by maintaining local state through the `remember` construct (line 3), storing the current model parameters and a logical training round counter. Each device then performs a local training step on its private dataset (line 5), producing an updated model and an associated training loss. To enable self-organized federation formation, devices compute a dynamic notion of distance based on model performance similarity. This is achieved through the function `loss_based_distances` (line 6), which derives a computational field reflecting how well neighboring models perform on local validation data. Leaders are then elected using `elect_leaders` (line 7), and a potential field toward each leader is computed via `distance_to` (line 8), implicitly defining coordination regions in a data-driven and adaptive manner. Within each coordination region, model updates are aggregated at the leader using a convergecast operation implemented through `collect_with` (line 9). In the example, local models are combined and averaged to obtain a regional aggregated model (line 10). The leader subsequently disseminates the updated model back to all devices in the region using `broadcast`(line 11), ensuring that federation members are synchronized for the next training iteration (line 12).

```
1  @aggregate
2  def client(initial_model_params):
3      set_value, value = remember((initial_model_params, 0))
4      local_model, tick = value
5      evolved_model, train_loss = local_training(local_model, training_data)
6      distances = loss_based_distances(evolved_model, validation_data)
7      leader = elect_leaders(threshold, distances)
8      potential = distance_to(leader, distances)
9      models = collect_with(potential, [evolved_model], lambda x, y: x + y)
10     aggregated_model = average_weights(models)
11     area_model = broadcast(leader, aggregated_model, distances)
12     set_value((area_model, tick + 1))
13     return potential
```

Listing 1.10: Implementation of SOFL [21] in Phyelds.

Integration with Third-Party Simulators. An additional relevant aspect of PHYELDS is that, although it provides a native simulator, it is designed to seamlessly integrate with external simulation frameworks by supplying appropriate bindings. This design enables PHYELDS to operate in heterogeneous simulation ecosystems without constraining the choice of the underlying simulator.

To demonstrate this capability, we developed an integration with the VMAS simulator [11], 1 which is widely used in the context of multi-agent reinforcement learning (MARL) and swarm robotics. Supporting an external simulator requires the implementation of two core components. The first component is a simulator-specific *runner* (in this case, a VMAS runner). At each simulated time step,

the runner retrieves the action computed by each agent, executes these actions within the selected simulator, and then provides the resulting observations or outcomes back to the agents. The second component is an *environment wrapper* for VMAS. This wrapper specifies how agents are initialized and how their state is updated at each simulation step. It is invoked by the runner and acts as an abstraction layer between Phyelds and the VMAS environment.

To showcase the functionality of the VMAS integration module, we implemented a simple aggregate program that reproduces flocking behavior according to the Vicsek model [34]. Each agent i is characterized by its position $\mathbf{r}_i(t)$ and by the angle $\Theta_i(t)$ defining the direction of its velocity at time t. At each discrete time step Δt, agents align their direction with that of their neighbors within a fixed interaction radius r, up to a noise term:

$$\Theta_i(t + \Delta t) = \langle \Theta_j \rangle_{\|\mathbf{r}_i - \mathbf{r}_j\| < r} + \eta_i(t), \tag{1}$$

where $\langle \Theta_j \rangle_{\|\mathbf{r}_i - \mathbf{r}_j\| < r}$ denotes the average direction of all agents (including i) located within distance r from agent i, and $\eta_i(t)$ represents a stochastic noise term.

After updating its direction, each agent moves at constant speed v according to

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + v\,\Delta t \begin{pmatrix} \cos \Theta_i(t) \\ \sin \Theta_i(t) \end{pmatrix}. \tag{2}$$

Listing 1.11 reports the aggregate program implementing the Vicsek flocking model within the Phyelds framework. Figure 4, instead, shows the temporal evolution of the environment as obtained through the native rendering capabilities of VMAS. As time progresses (from left to right), agents progressively align their directions, eventually exhibiting coherent and collective motion.

```
@aggregate
def action():
    myself = sense('agent')
    vel = myself.state.vel.squeeze()
    neighbors_info = neighbors(vel).exclude_self()
    velocities = [vel for vel in neighbors_info.values()]
    avg_vel = mean_velocity(velocities)
    theta = velocity_to_angle(vel, avg_vel)
    noise = perturbation()
    theta = theta + 0.1 * noise
    next_vel = [torch.cos(theta).item(), torch.sin(theta).item()]
    store("action", next_vel)
```

Listing 1.11: Implementation of the Vicsek Model in Phyelds using VMAS.

5 Conclusion

This paper introduces Phyelds, a Pythonic framework embodying a self-contained implementation of the field calculus within the Python ecosystem. By combining the formal foundations of aggregate programming with an imperative and object-oriented API, Phyelds lowers the barrier to adoption for data science and machine learning practitioners.

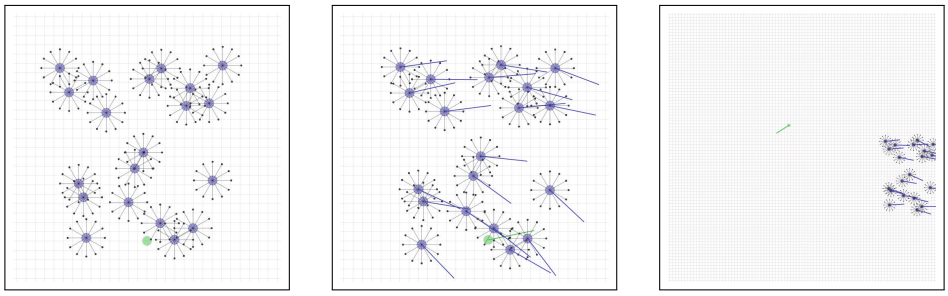


Fig. 4: Vicsek flocking simulation in VMAS using PHYELDS. Agents (cirlces) align their velocities over time (left to right), exhibiting coherent collective motion.

Its modular architecture supports both core field calculus operators and higher-level coordination patterns, while enabling seamless integration with machine learning libraries and external simulators such as VMAS. The presented examples demonstrate the expressiveness of the framework across classical aggregate computing patterns, self-organizing federated learning, and multi-agent coordination scenarios.

Future work will focus on performance and scalability improvements, deeper integration with mainstream ML toolchains, systematic benchmarking against existing frameworks, integration with other third-party simulators (e.g., [35,23]) and deployment in real-world edge and robotic systems (e.g., [2,6]) to further validate the practical potential of Python-based aggregate programming.

References

1. Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D.G., Steiner, B., Tucker, P.A., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., Zheng, X.: Tensorflow: A system for large-scale machine learning. In: Keeton, K., Roscoe, T. (eds.) 12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016. pp. 265–283. USENIX Association (2016), `https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi`
2. Aguzzi, G., Bacchini, L., Baiardi, M., Casadei, R., Cortecchia, A., Domini, D., Farabegoli, N., Pianini, D., Viroli, M.: A demonstrator for self-organizing robot teams. In: Giusto, C.D., Ravara, A. (eds.) Coordination Models and Languages - 27th IFIP WG 6.1 International Conference, COORDINATION 2025, Held as Part of the 20th International Federated Conference on Distributed Computing Techniques, DisCoTec 2025, Lille, France, June 17-19, 2025, Proceedings. Lecture Notes in Computer Science, vol. 15731, pp. 230–244. Springer (2025). `https://doi.org/10.1007/978-3-031-95589-1_12`, `https://doi.org/10.1007/978-3-031-95589-1_12`
3. Aguzzi, G., Casadei, R., Viroli, M.: Addressing collective computations efficiency: Towards a platform-level reinforcement learning approach. In: 2022 IEEE Inter-

national Conference on Autonomic Computing and Self-Organizing Systems (ACSOS). pp. 11–20. IEEE (2022)
4. Aguzzi, G., Casadei, R., Viroli, M.: Machine learning for aggregate computing: a research roadmap. In: 2022 IEEE 42nd International Conference on Distributed Computing Systems Workshops (ICDCSW). pp. 119–124. IEEE (2022)
5. Aguzzi, G., Viroli, M., Esterle, L.: Field-informed reinforcement learning of collective tasks with graph neural networks. In: 2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS). pp. 37–46. IEEE (2023)
6. Andruccioli, M., Cortecchia, A., Domini, D., Farabegoli, N., Delnevo, G., Pianini, D., Venanzi, R., Viroli, M.: Harmonikt: a unifying middleware for heterogeneous robot fleets. In: 23rd Consumer Communications & Networking Conference, CCNC 2026, Las Vegas, NV, USA, January 9-12, 2026. pp. 1–6. IEEE (2026). https://doi.org/10.1109/CCNC65079.2026.11366553, https://doi.org/10.1109/CCNC65079.2026.11366553
7. Audrito, G., Torta, G.: FCPP to aggregate them all p. 103026 (2024). https://doi.org/10.1016/J.SCICO.2023.103026, https://doi.org/10.1016/j.scico.2023.103026
8. Audrito, G., Viroli, M., Damiani, F., Pianini, D., Beal, J.: A higher-order calculus of computational fields (1), 5:1–5:55 (2019). https://doi.org/10.1145/3285956, https://doi.org/10.1145/3285956
9. Aurenhammer, F.: Voronoi diagrams - A survey of a fundamental geometric data structure. ACM Comput. Surv. **23**(3), 345–405 (1991). https://doi.org/10.1145/116873.116880, https://doi.org/10.1145/116873.116880
10. Beal, J., Pianini, D., Viroli, M.: Aggregate programming for the internet of things (9), 22–30 (2015). https://doi.org/10.1109/MC.2015.261, https://doi.org/10.1109/MC.2015.261
11. Bettini, M., Kortvelesy, R., Blumenkamp, J., Prorok, A.: VMAS: A vectorized multi-agent simulator for collective robot learning. In: Bourgeois, J., Paik, J., Piranda, B., Werfel, J., Hauert, S., Pierson, A., Hamann, H., Lam, T.L., Matsuno, F., Mehr, N., Makhoul, A. (eds.) Distributed Autonomous Robotic Systems - 16th International Symposium, DARS 2022, Montbéliard, France, 28-30 November 2022. Springer Proceedings in Advanced Robotics, vol. 28, pp. 42–56. Springer (2022). https://doi.org/10.1007/978-3-031-51497-5_4, https://doi.org/10.1007/978-3-031-51497-5_4
12. Casadei, R.: Macroprogramming: Concepts, state of the art, and opportunities of macroscopic behaviour modelling (13s), 275:1–275:37 (2023). https://doi.org/10.1145/3579353, https://doi.org/10.1145/3579353
13. Casadei, R., Aguzzi, G., Audrito, G., Damiani, F., Pianini, D., Scarso, G., Torta, G., Viroli, M.: Software engineering for collective cyber-physical ecosystems (Jan 2025). https://doi.org/10.1145/3712004, https://doi.org/10.1145/3712004, just Accepted
14. Casadei, R., Pianini, D., Viroli, M., Natali, A.: Self-organising coordination regions: A pattern for edge computing. In: Nielson, H.R., Tuosto, E. (eds.) Coordination Models and Languages - 21st IFIP WG 6.1 International Conference, COORDINATION 2019, Held as Part of the 14th International Federated Conference on Distributed Computing Techniques, DisCoTec 2019, Kongens Lyngby, Denmark, June 17-21, 2019, Proceedings. Lecture Notes in Computer Science, vol. 11533, pp. 182–199. Springer (2019). https://doi.org/10.1007/978-3-030-22397-7_11, https://doi.org/10.1007/978-3-030-22397-7_11

15. Casadei, R., Viroli, M., Aguzzi, G., Pianini, D.: Scafi: A scala DSL and toolkit for aggregate programming p. 101248 (2022). https://doi.org/10.1016/J.SOFTX.2022.101248, https://doi.org/10.1016/j.softx.2022.101248
16. Cortés, J., Martínez, S., Karatas, T., Bullo, F.: Coverage control for mobile sensing networks. IEEE Trans. Robotics Autom. **20**(2), 243–255 (2004). https://doi.org/10.1109/TRA.2004.824698, https://doi.org/10.1109/TRA.2004.824698
17. Domini, D., Aguzzi, G., Esterle, L., Viroli, M.: FBFL: A field-based coordination approach for data heterogeneity in federated learning. Logical Methods in Computer Science **22**, 30 (2026). https://doi.org/10.46298/lmcs-22(1:19)2026, https://lmcs.episciences.org/17663
18. Domini, D., Aguzzi, G., Viroli, M.: Profed: a benchmark for proximity-based non-iid federated learning. Journal of Open Research Software **14** (2026). https://doi.org/10.5334/jors.624, https://openresearchsoftware.metajnl.com/articles/10.5334/jors.624
19. Domini, D., Cavallari, F., Aguzzi, G., Viroli, M.: Scarlib: Towards a hybrid toolchain for aggregate computing and many-agent reinforcement learning. Sci. Comput. Program. **238**, 103176 (2024). https://doi.org/10.1016/J.SCICO.2024.103176, https://doi.org/10.1016/j.scico.2024.103176
20. Domini, D., Erhan, L., Aguzzi, G., Cavallaro, L., Zenoozi, A.D., Liotta, A., Viroli, M.: Sparse self-federated learning for energy efficient cooperative intelligence in society 5.0. In: International Joint Conference on Neural Networks, IJCNN 2025, Rome, Italy, June 30 - July 5, 2025. pp. 1–8. IEEE (2025). https://doi.org/10.1109/IJCNN64981.2025.11228400, https://doi.org/10.1109/IJCNN64981.2025.11228400
21. Domini, D., Farabegoli, N., Aguzzi, G., Viroli, M., Esterle, L.: Decentralized proximity-aware clustering for collective self-federated learning. Internet of Things **35**, 101841 (2026). https://doi.org/10.1016/J.IOT.2025.101841, https://doi.org/10.1016/j.iot.2025.101841
22. Heinzelman, W.R., Chandrakasan, A.P., Balakrishnan, H.: Energy-efficient communication protocol for wireless microsensor networks. In: 33rd Annual Hawaii International Conference on System Sciences (HICSS-33), 4-7 January, 2000, Maui, Hawaii, USA. IEEE Computer Society (2000). https://doi.org/10.1109/HICSS.2000.926982, https://doi.org/10.1109/HICSS.2000.926982
23. ter Hoeven, E., Kwakkel, J., Hess, V., Pike, T., Wang, B., Rht, Kazil, J.: Mesa 3: Agent-based modeling with python in 2025. J. Open Source Softw. **10**(109), 7668 (2025). https://doi.org/10.21105/JOSS.07668, https://doi.org/10.21105/joss.07668
24. Macenski, S., Foote, T., Gerkey, B.P., Lalancette, C., Woodall, W.: Robot operating system 2: Design, architecture, and uses in the wild. Sci. Robotics **7**(66) (2022). https://doi.org/10.1126/SCIROBOTICS.ABM6074, https://doi.org/10.1126/scirobotics.abm6074
25. McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In: Singh, A., Zhu, X.J. (eds.) Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS 2017, 20-22 April 2017, Fort Lauderdale, FL, USA. Proceedings of Machine Learning Research, vol. 54, pp. 1273–1282. PMLR (2017), http://proceedings.mlr.press/v54/mcmahan17a.html
26. McMahan, H.B., Moore, E., Ramage, D., y Arcas, B.A.: Federated learning of deep networks using model averaging. CoRR **abs/1602.05629** (2016), http://arxiv.org/abs/1602.05629

27. Mo, Y., Audrito, G., Dasgupta, S., Beal, J.: A resilient leader election algorithm using aggregate computing blocks**supported by the defense advanced research projects agency (darpa) under contract no. hr001117c0049. the views, opinions, and/or findings expressed are those of the author(s) and should not be interpreted as representing the official views or policies of the department of defense or the u.s. government. this document does not contain technology or technical data controlled under either u.s. international traffic in arms regulation or u.s. export administration regulations. approved for public release, distribution unlimited (darpa distar case 32200, 10/31/19). mo was also partially supported by the australian research council under grant dp190100887 and dp160104500. IFAC-PapersOnLine **53**(2), 3336–3341 (2020). https://doi.org/https://doi.org/10.1016/j.ifacol.2020.12.1497, https://www.sciencedirect.com/science/article/pii/S2405896320319637, 21st IFAC World Congress

28. Mo, Y., Audrito, G., Dasgupta, S., Beal, J.: Near-optimal knowledge-free resilient leader election. Autom. **146**, 110583 (2022). https://doi.org/10.1016/J.AUTOMATICA.2022.110583, https://doi.org/10.1016/j.automatica.2022.110583

29. Mo, Y., Beal, J., Dasgupta, S.: An aggregate computing approach to self-stabilizing leader election. In: 2018 IEEE 3rd International Workshops on Foundations and Applications of Self* Systems (FAS*W). pp. 112–117. IEEE (2018). https://doi.org/10.1109/FAS-W.2018.00034

30. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E.Z., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: Pytorch: An imperative style, high-performance deep learning library. In: Wallach, H.M., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E.B., Garnett, R. (eds.) Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada. pp. 8024–8035 (2019), https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

31. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in python. J. Mach. Learn. Res. **12**, 2825–2830 (2011). https://doi.org/10.5555/1953048.2078195, https://dl.acm.org/doi/10.5555/1953048.2078195

32. Pianini, D., Casadei, R., Viroli, M., Natali, A.: Partitioned integration and coordination via the self-organising coordination regions pattern pp. 44–68 (2021). https://doi.org/10.1016/J.FUTURE.2020.07.032, https://doi.org/10.1016/j.future.2020.07.032

33. Pianini, D., Viroli, M., Beal, J.: Protelis: practical aggregate programming. pp. 1846–1853. ACM (2015). https://doi.org/10.1145/2695664.2695913, https://doi.org/10.1145/2695664.2695913

34. Vicsek, T., Czirók, A., Ben-Jacob, E., Cohen, I., Shochet, O.: Novel type of phase transition in a system of self-driven particles. Physical review letters **75**(6), 1226 (1995)

35. Zinoviev, D.: Discrete event simulation: It's easy with simpy! CoRR **abs/2405.01562** (2024). https://doi.org/10.48550/ARXIV.2405.01562, https://doi.org/10.48550/arXiv.2405.01562